



This Exploratory Data Analysis (EDA) assignment is designed to test data manipulation, visualization, and insight extraction skills. Students should select a unique dataset from Kaggle to ensure diverse submissions.

Assignment: Comprehensive Exploratory Data Analysis

Objective

Perform a deep-dive analysis on a raw dataset to uncover patterns, handle inconsistencies, and prepare the data for potential machine learning modeling.

Deliverables

1. A Jupyter Notebook (.ipynb) or a Python script (.py).
2. Dataset Source: Provide the Kaggle URL in the first cell/comments.
3. Documentation: Clear markdown cells or comments explaining why a specific step was taken.

Required Technical Tasks

Students must include the following functional sections in their submission:

1. Environment Setup & Data Loading



- Import necessary libraries: pandas, numpy, matplotlib.pyplot, and seaborn.
- Load the dataset and display the first 5–10 rows using `.head()`.

2. Structural Inspection

Perform an initial audit of the data structure using these functions:

- `df.info()`: Check data types and memory usage.
- `df.describe()`: Generate descriptive statistics for numerical columns.
- `df.shape`: Report the number of rows and features.
- `df.columns`: List all feature names.

3. Data Cleaning (The Preprocessing Phase)

- Missing Value Analysis: Identify nulls using `df.isnull().sum()` and decide whether to drop or impute them.
- Duplicate Handling: Check for and remove duplicate rows using `df.duplicated().sum()`.
- Data Type Correction: Convert "object" types to "datetime" or "category" where appropriate (e.g., converting a date string to a usable format).



4. Univariate Analysis

Analyze individual variables to understand their distribution:

- Numerical: Use `sns.histplot()` or `sns.boxplot()` to identify skewness and outliers.
- Categorical: Use `sns.countplot()` or `.value_counts()` to see the frequency of classes.

5. Bivariate & Multivariate Analysis

Explore relationships between different features:

- Correlation Matrix: Use `df.corr()` visualized with `sns.heatmap()` to find dependencies.
- Scatter Plots: Use `sns.scatterplot()` to check relationships between two numerical variables.
- Box Plots (Grouped): Use `sns.boxplot(x='category', y='numeric')` to see how values differ across groups.

6. Final Summary & Insights

- Write a brief conclusion summarizing three key takeaways from the data.
- State if the data is "model-ready" (e.g., mention if features need scaling or encoding).

